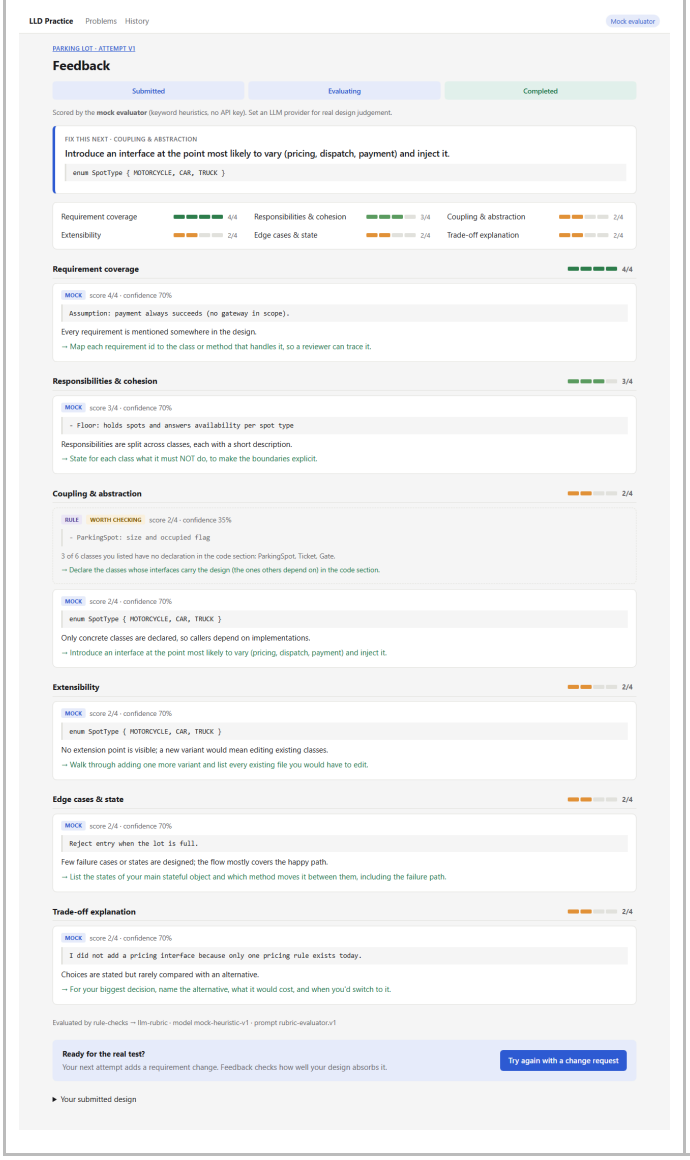
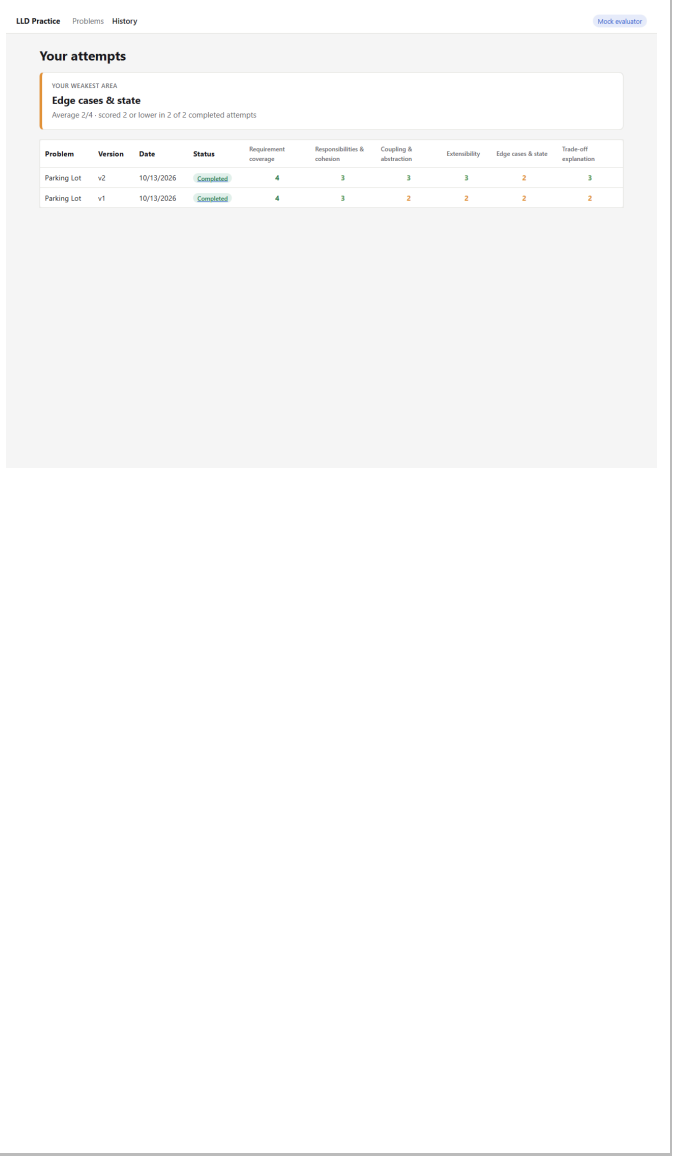
LLD Practice Platform

Practise Low-Level Design the way it is tested: pick a problem, write a structured design, and get feedback that **scores it against the requirements and a fixed rubric, not one "correct" answer**. Every point quotes your own words, and code drops any point whose quote isn't really in your submission. Your second attempt adds a **requirement change** (e.g. "add EV charging billed per kWh"), because surviving a change is the real LLD skill. History shows which rubric dimension you keep getting wrong.

Feedback (scores, top fix, quoted findings)	History with weakest area
	

More: problems · editor · v2 change request · failed with partial feedback. Notes: Research · Design · AI usage.

Quick start (no API key needed)

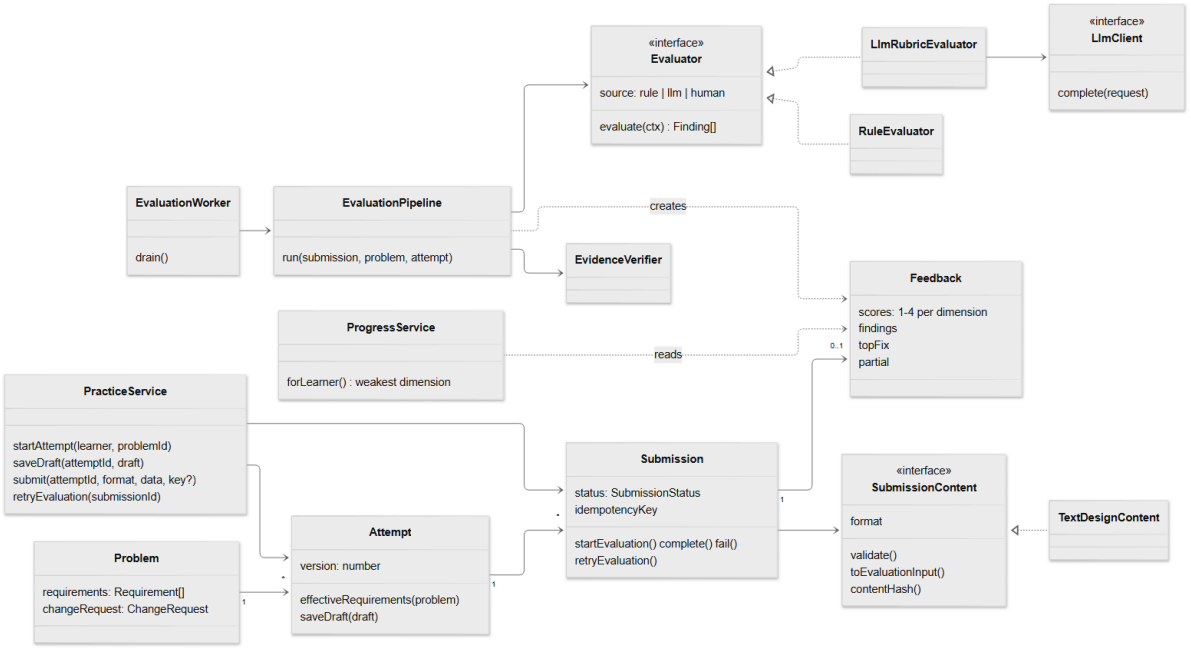
```
npm install && npm run dev        # API on :3000, UI on http://localhost:5173
npm test                          # 37 tests
npm run build && npm start        # production: everything on http://localhost:3000
```

Requires Node 22.13+ (uses the built-in `node:sqlite`, so there is no native build step). With no configuration it uses the **mock evaluator**: deterministic keyword heuristics behind the real prompt → zod → evidence-check path, labelled "Mock evaluator" in the UI.

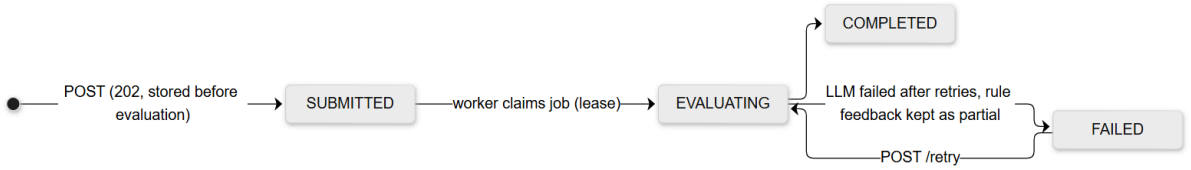
Use a real LLM: copy `.env.example` to `.env` and set one of:

```
LLM_PROVIDER=anthropic  ANTHROPIC_API_KEY=sk-ant-...          # model defaults to claude-sonnet-5-5
LLM_PROVIDER=openai     OPENAI_API_KEY=sk-...  LLM_MODEL=gpt-4.1
```

Architecture



diagram



diagram

The practice flow (`PracticeService`, `Attempt`, `Submission`) never knows **which format** was submitted (`SubmissionContent`) or **which evaluator** produced feedback (`Evaluator`). Those are the only two Strategy interfaces. Evaluation is a pipeline: rules first, and a blocking structural finding stops it before the LLM is called. Then the LLM scores the design against the rubric, and `EvidenceVerifier` drops any finding whose quote is not in the submission.

Key decisions

- **Rubric, not reference answer:** 6 fixed dimensions, scored 1–4 with an anchor sentence per score, plus a per-problem requirement checklist.
- **Evidence or it didn't happen:** findings must quote the submission verbatim, and code verifies the quote. The top fix is chosen by code from verified findings, not written freely by the LLM.
- **Deterministic first:** structure, class/code consistency, god class and keyword gaps are checked by code. Keyword gaps go to the LLM as hints to confirm or reject.
- **Versioned prompt + structured output:** `src/prompts/rubric-evaluator.v1.md`, validated with zod, with one repair retry. Learner text is escaped so it cannot leave `<submission>`.
- **Never lose work:** the submission and its job are stored in one transaction before evaluation. Requests are idempotent, jobs are leased, failures keep partial rule feedback, and the learner can

retry.

- **First thing to split out as usage grows: the evaluation worker** (its own process on the same jobs table, then a real queue), because LLM calls are the slow, costly, bursty part.

Decision	Gain	Cost
Structured text vs diagrams	Shows reasoning + code-level abstraction; cheap to build	Relationships inferred from text
1–4 rubric vs 100-point score	Repeatable; says what to work on	Coarse; small gains don't move a score
Drop unquoted findings	No invented criticism	Correct paraphrased points are lost
SQLite as job queue	Survives restarts, zero infra	Single process; up to 2 s polling latency
Transition table vs State pattern	All legal moves in one place	Retry-from-FAILED needs one explicit rule
Mock evaluator by default	Zero-setup demo; exercises the whole pipeline	Heuristics, not real judgement

The design questions

1. **What must a learner provide?** Five sections: requirements and assumptions, classes and responsibilities, key interfaces as code, main flow, and trade-offs. Each has a minimum length, enforced with a 400 that lists every incomplete section.
2. **Useful feedback when many designs are valid?** Judge against requirements + rubric anchors, never a model answer. Give a score per dimension, findings that quote the learner's own text, and one fix to do next.
3. **Deterministic vs LLM?** Code checks anything with one right answer: structure, lengths, declared types, method counts, keyword presence, state transitions and idempotency. The LLM judges design quality and confirms or rejects the rule hints.
4. **New evaluation approach or format?** Implement `Evaluator` or `SubmissionContent` and register it. Change tests A and B do exactly this in code.
5. **Slow or failing evaluation?** 202 plus a background worker, a 60 s timeout with 2 retries, leases, partial feedback, and a manual retry (see the state diagram above).

Change tests (`tests/change-tests.test.ts`)

Change	Added	Untouched
A: class-diagram submissions	`FakeDiagramContent implements SubmissionContent` + 1 registry line	Attempt, Submission, PracticeService, pipeline, evaluators, worker, DB schema
B: human/rule reviewer	`FakeHumanEvaluator implements Evaluator` + 1 list entry	PracticeService, worker, Submission, Feedback (findings carry `source: "human"`)

Limitations and next steps

- Keyword coverage is crude: synonyms are missed and substrings can match, so it only produces low-confidence hints.
- LLM scores can still vary slightly between runs; temperature 0 isn't available on current Claude models, so consistency relies on anchors, the schema and code-side scoring.

- One demo learner, no auth. Repositories are synchronous (Node's built-in `node:sqlite`).
- **Next:** a small eval set of real designs to calibrate the prompt; human review as a third evaluator; diagram input; per-requirement traceability in feedback.

Project structure

```
src/domain/          pure domain: Problem, Attempt, Submission, content + evaluation interfaces,
pipeline, Feedback, progress
src/evaluators/      RuleEvaluator, LlmRubricEvaluator (zod + repair), EvidenceVerifier
src/llm/             LlmClient + Anthropic / OpenAI / mock clients, timeout+retry helper
src/prompts/         rubric-evaluator.v1.md (versioned prompt) + builder
src/infra/           SQLite migrations, repositories, job queue; JSON problem repository
src/worker/          EvaluationWorker (lease-based, in-process)
src/api/             Express routes, DTOs, error mapping   ·  src/container.ts: composition root
src/data/problems/   4 seeded problems with requirements + change requests
web/                 React + Vite UI
tests/               Vitest: domain, evaluators, worker failure paths, change tests, API e2e
```

API: `GET /api/problems[/:id]` · `POST /api/attempts` · `GET /api/attempts/:id` · `PUT /api/attempts/:id/draft` · `POST /api/attempts/:id/submissions` (`Idempotency-Key`; 202/200/400) · `GET /api/submissions/:id` · `POST /api/submissions/:id/retry` · `GET /api/history[?problemId=]`. Errors are always `{ error: { code, message, details? } }`.

AI Usage

I used AI as a design sparring partner and as an implementer, and kept the decisions (and the checking) for myself. In the research and design session I used Claude to critique options. I then wrote a detailed build brief (scope, domain model, rubric, failure handling, tests) and had Claude Code implement it phase by phase while I reviewed the output. Every decision below is also in the running log, `docs/ai-log.md`.

Five decisions

1. Grade against a reference solution, or against requirements + a rubric?

- *AI suggested:* the obvious first design compared the learner's classes with a model solution (similar to AlgoMaster's per-stage reference grading).
- *I chose:* a per-problem requirement checklist plus a fixed 6-dimension rubric.
- *Why:* LLD has many valid answers. Reference matching punishes a correct design that differs from the reference, and teaches copying. The rubric judges qualities (cohesion, coupling, extensibility) that any valid design can show.

2. One 100-point AI score, or 1–4 per dimension with anchors?

- *AI suggested:* an overall score with a short explanation.
- *I rejected it:* a single number gives no direction, and an LLM's 73 vs 78 is noise.
- *Built:* a 1–4 score per dimension, each score with a written anchor sentence (`rubric.ts`). The scores are computed in code from the findings, and so is the "fix this next".

3. Trust the LLM's findings, or verify them?

- *AI suggested:* asking the model to "cite evidence" in the prompt.
- *I accepted, plus a code check:* `EvidenceVerifier` drops any finding whose quote is not actually in the submission (case and whitespace insensitive). The top fix is also picked by code from verified findings, rather than written by the LLM.
- *Trade-off I accepted:* a correct point the model paraphrased instead of quoting is lost.

4. State pattern or transition table for submission status?

- *AI suggested:* the State pattern, a common textbook answer for status workflows.
- *I chose:* a transition table.
- *Why:* SUBMITTED/EVALUATING/COMPLETED/FAILED have no behaviour of their own, only rules about which may follow which. A State class per status would be a pattern added to show off.
- *Verification:* the AI-written tests caught a real bug in the AI-written code. Because the table allows SUBMITTED → EVALUATING, "retry" was accepted on a submission that had never failed. Retry is now an explicit FAILED-only rule.

5. Text, diagram and code, or structured text only?

- *AI suggested:* supporting all three for "more evidence".
- *I chose:* structured text in five sections, one of them code.
- *Why:* it is the smallest format that shows both reasoning and abstraction, and a diagram editor would have eaten the time budget. To show diagrams can be added later, change test A runs a fake class-diagram format through the unchanged flow.

How I worked with AI

- **Separate roles.** I used AI for design critique (asking it to argue against a design) and, separately, for implementation against a written brief. The AI never decided scope.

- **Structured output, not open questions.** The evaluator prompt is versioned (`rubric-evaluator.v1.md`), fixes the rubric with anchors, tags every input, and demands JSON. That JSON is validated with zod and gets one repair retry.
- **Adversarial checks.** The prompt treats the submission as untrusted data, and code escapes delimiter tags, which a test proves. I asked the AI to write failure-case tests: timeouts, invalid JSON, lease expiry and injection. These tests, not just happy paths, are how I accepted the code.
- **Verify against reality, not memory.** The brief asked for temperature 0. Before coding, the current Claude API reference showed that current Claude models reject non-default sampling parameters, so the Anthropic client omits it. Consistency comes from the anchors, the schema and code-side scoring. This deviation is noted in the design note.
- **Every AI output gets a check:** tests (37), quote verification, schema validation, and a manual run of the full flow, including a forced failure with a bad API key.